The key benefits of smart contracts are that they are transparent, secure, and self-executing, which can help to reduce the costs and risks associated with traditional contract processes. Additionally, because they exist on a decentralized network, they are resistant to tampering or interference, making them a more secure and trustworthy way of conducting transactions.

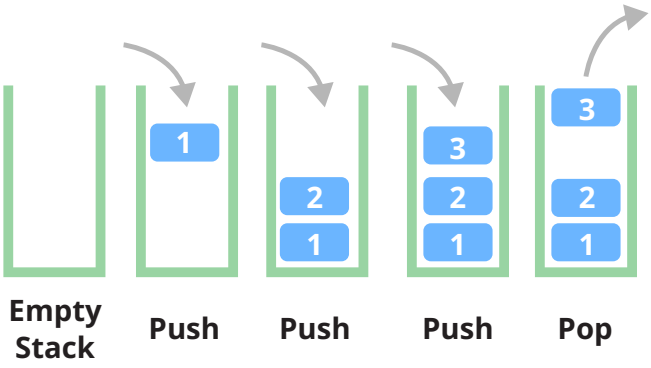
In a similar way, ***Bitcoin*** uses script to make sure that specific conditions are met before a transaction is processed.

While other *blockchain* networks, such as Ethereum, also support smart contracts and programmable transactions, they use different programming languages and approaches to enforce the rules and conditions of transactions. Only Bitcoin uses script.

Script is a very basic programming language, but it's powerful enough to handle a wide range of transactions. For example, it can be used to create **multi-signature** transactions, where multiple people must sign off on a transaction before it can be processed, or to create a smart contract, where a transaction is automatically executed when certain conditions are met.

While script may seem complex, the basic idea behind it is actually quite simple. By using script, the Bitcoin network can automatically enforce the rules and conditions of transactions, making it a secure and efficient way to transfer value.

How Scripting is Used in Bitcoin Transactions



- Imagine you have a row of coins and you want to sort them into different slots, like putting coins into different piggy banks. The order in which you put the coins into the slots is important. This is similar to how satoshis are transferred in a transaction. The inputs are like a row of coins and the outputs are the slots waiting to receive a coin. To assign the coins to the slots, you go through each coin in the row in order, and put each one into the first available slot. This is called "first-in-first-out" or FIFO, which means that the first coin in the row goes into the first available slot, and so on.

- Scripts operate on a **stack-based system**, where instructions are processed in the order they appear, from top to bottom.The concept is similar to a stack of plates, where you can only access the plate that is on top of the stack.